

Expresiones Regulares

**Programación Concurrente
2021**

Ing. Ventre, Luis O.

Expresiones Regulares



- Que son? Y para que sirven?
- Es una descripción compacta que representa un lenguaje regular.
- Son utilizadas para expresar patrones de búsqueda (combinaciones de caracteres) dentro de cadenas.

Expresiones Regulares



- Meta-caracteres y su significado

[] ^ { } . | ? * + () \ \$

- Son caracteres especiales los cuales dentro de las expresiones no se interpretan literalmente. Tienen significados especiales.

- Meta-caracteres y su significado
- [] Corchetes:
 - Proveen una lista potencial de caracteres coincidentes en una posición del texto analizado.

Ejemplos

[abc] Alguno de los caracteres a, b, c

[a-z] Cualquier carácter entre la a y la z

7[Pp][Mm]

[^ab] un carácter que NO sea ni a ni b

Expresiones Regulares

- $\{ \}$ Llaves: representan repetición de caracteres
 - $\{n\}$: n veces.
 - $\{n, \}$: como mínimo n veces, sin limite superior de veces.
 - $\{n, m\}$: entre n y m veces.

Ejemplo

$a\{4\}$: representa la cadena 'aaaa'

Expresiones Regulares

- Meta-caracteres y su significado
- El . (punto):
 - Representa cualquier carácter individual a excepción de «nueva línea», comodín.
- | alternativa:
 - Representa la función lógica or, o alternativa entre opciones.
- () Paréntesis:
 - Represente grupos

Expresiones Regulares



- Cuantificadores:

Cuantificador	Match	Equivalente a
?	Cero ó una vez	$\{ 0 , 1 \}$
*	Cero ó más veces	$\{ 0 , \}$
+	Una ó más veces	$\{ 1 , \}$

Ejemplos:

$(abc)^+$: representa 1 «ó» más veces la cadena `abc`

$(a|b)^*$: representa cero «ó» más veces a «ó» b

Expresiones Regulares

- Greedy vs Lazy
- En los patrones de búsqueda, es posible en nuestra expresión regular indicar dos comportamientos diferentes: codicioso o vago.
- Los cuantificadores * + ? son codiciosos, coinciden con la mayor cantidad de texto posible
 - Ejemplo: si utilizamos `<.*>` en el string `'<a>b<c>'`
Coincidirá con toda la cadena

Expresiones Regulares



- Greedy vs Lazy Modificar el comportamiento codicioso
- El carácter especial ?, es un cuantificador por si mismo. Pero al colocarlo luego de un cuantificador cambia su comportamiento a **lazy o minimal**.
- En el string ejemplo:

`'<a>b<c>'`

El patron `<.*?>` coincidirá con `<a>`

Expresiones Regulares

- Como ver los ejemplos gráficamente:

<https://www.debuggex.com/>

DebuggexBeta Pricing Blog Tutorial Login/Signup

Share <> Code Snippet

Using regexes for extracting data from web pages? Check out [ParseHub](#), a visual web scraping tool built by the team behind Debuggex.

Untitled Regex No description Embed on StackOverflow



JavaScript View Cheatsheet Flags

1 `<.*?>`

Result: **Matches** starting at the black triangle slider

1 `<a>cualquiercosa<b>`

START POSITION

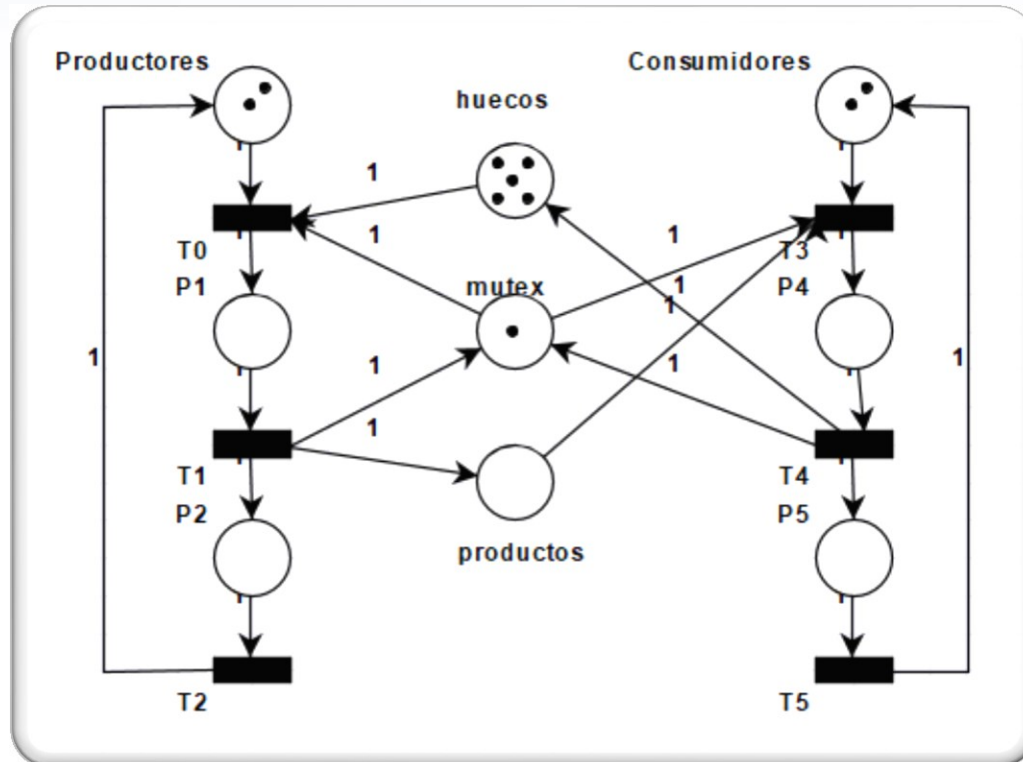
Expresiones Regulares

- Expresiones regulares en Python
- <https://docs.python.org/es/3/library/re.html>
- En este link, encontrarán mucha info sobre sintaxis y funciones en Python para manejar expresiones regulares.
- Ver funciones:
re.match re.search **re.sub** **re.subn**

Expresiones Regulares y RdP

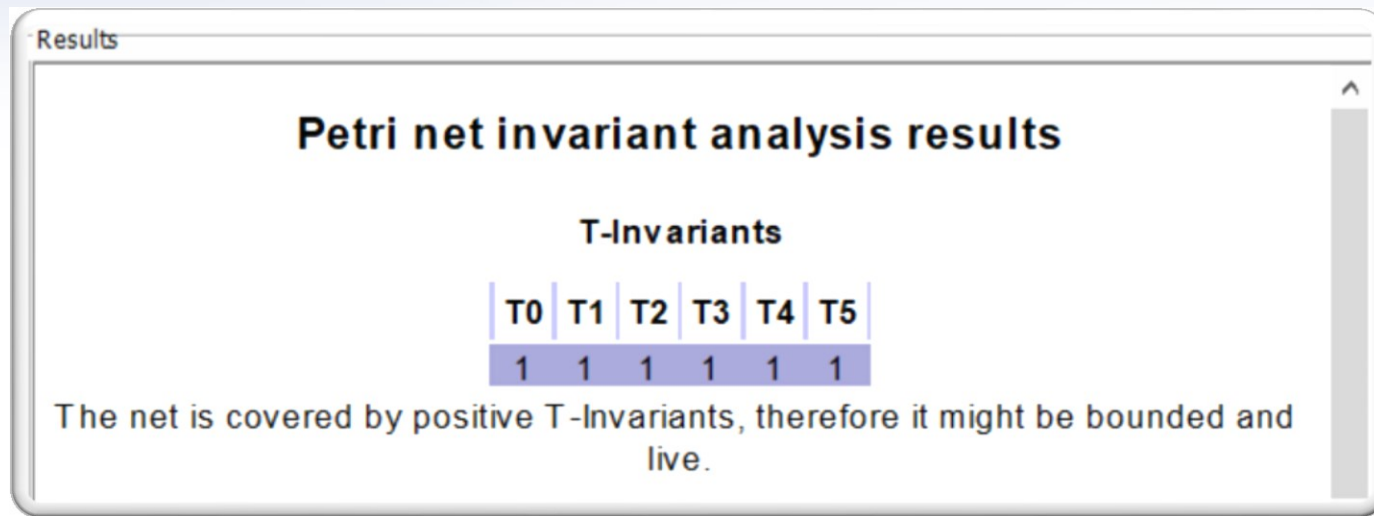


- Análisis dinámicos de redes de Petri:
 - Productor Consumidor Buffer Acotado Múltiples Agentes



Expresiones Regulares y RdP

- Invariantes de transición de esta red?



- Cómo puedo asegurarme que luego de una ejecución de n transiciones, una red de petri siempre cumplió con sus T-invariantes.

Expresiones Regulares y RdP

- Script para analizar los logs:
 - Es necesario realizar un script en el lenguaje seleccionado, por ej. Python, que lea de un archivo txt las líneas de caracteres y las procese con una expresión regular.
 - **IMPORTANTE:** Si la RdP tiene más de un invariante de transición, deben agruparse en una sola expresión regular.

Expresiones Regulares y RdP

- Mas de un invariante...expresión regular única?
- Suponga una red con 2 invariantes:
 $T_0 T_1 T_2$ y $T_0 T_2 T_3$.
 - Estos invariantes comparten su transición inicial

- Mas de un invariante...expresión regular única?

$T_0 T_1 T_2$ y $T_0 T_2 T_3$.

- Si hago una regex para buscar el T_{inv1} y luego otra regex para buscar el T_{inv2} . Corro el riesgo de asignar como coincidencia o match de un invariante transiciones que pertenecían al otro invariante.
- Supongamos en el siguiente LOG sintético :

$T_0 T_1 T_0 T_2 T_0 T_1 T_0 T_2 T_2 T_3 T_2 T_3$

CONCLUSION: 1 sola REGEX con todos los posibles caminos (invariantes o combinaciones de ellos)

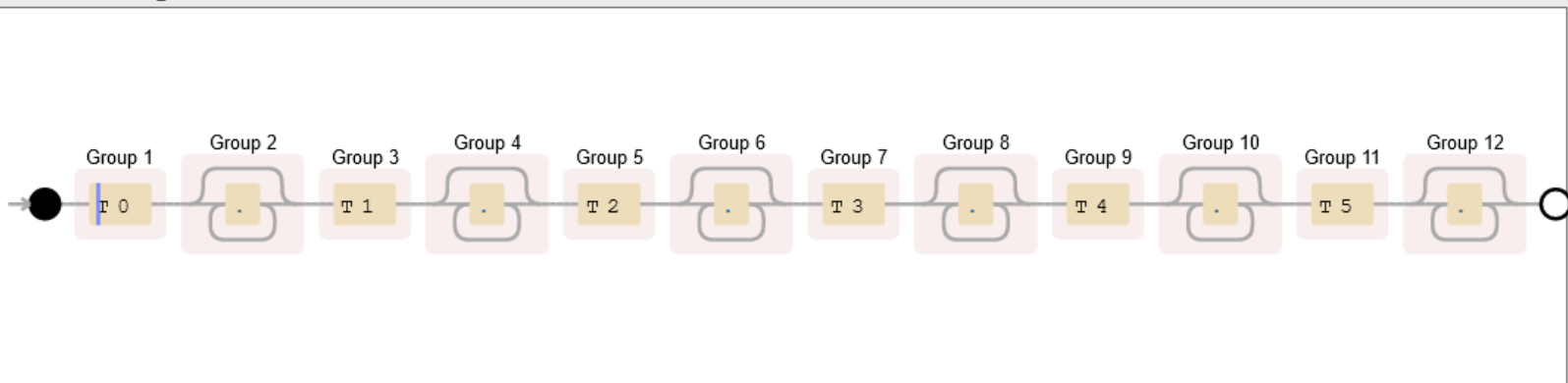
Expresiones Regulares y RdP

- Comencemos el análisis: veamos una ejecución

Expresiones Regulares y RdP

- Regex para analizarlo?

Untitled Regex No description [Embed on StackOverflow](#)



Python [View Cheatsheet](#) Flags

```
1 (T0) (.*?) (T1) (.*?) (T2) (.*?) (T3) (.*?) (T4) (.*?) (T5) (.*?)
```

Result: Matches starting at the black triangle slider

```
1 T0T1T2T3T4T5
```

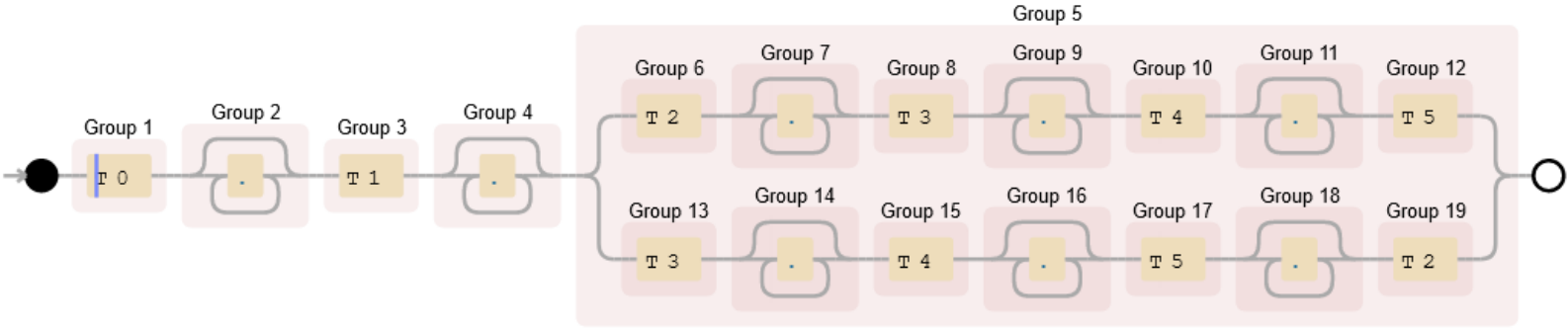
START POSITION

- Es posible alguna otra combinación del mismo invariante?

Expresiones Regulares y RdP

- Regex para analizarlo:

Untitled Regex No description [Embed on StackOverflow](#)



Python [View Cheatsheet](#) [Flags](#)

```
1 (T0) (.*) (T1) (.*) ((T2) (.*) (T3) (.*) (T4) (.*) (T5) | (T3) (.*) (T4) (.*) (T5) (.*) (T2))
```

Result: Matches starting at the black triangle slider

```
1 T0T1T2T3T4T5
```

- Es posible alguna otra combinación del mismo invariante?

Expresiones Regulares y RdP

- Regex para analizarlo:

Untitled Regex No description [Embed on StackOverflow](#)

Python [View Cheatsheet](#) Flags

```
1 (T0) (.*?) (T1) (.*?) ((T2) (.*?) (T3) (.*?) (T4) (.*?) (T5) | (T3) (.*?) (T4) (.*?) ((T5) (.*?) (T2) | (T2) (.*?) (T5))) |
```

Result: Matches starting at the black triangle slider

```
1 T0T1T2T3T4T5T0T1T3T4T5T2
```

Expresiones Regulares y RdP



- No de nuevo!

Expresiones Regulares y RdP

Untitled Regex No description [Embed on StackOverflow](#)

Python [View Cheatsheet](#) [Flags](#)

```
1 (T0) (.*?) (T1) (.*?) ((T2) (.*?) (T3) (.*?) (T4) (.*?) (T5) | ((T3) (.*?) ((T2) (.*?) (T4) (.*?) (T5) | (T4) (.*?) ((T2) (.*?) (T5) | ((T5) (.*?) (T2))))))
```

Result: Matches starting at the black triangle slider

```
1 T0T1T3T4T5T2T0T1T3T4T5T2
```


Páginas útiles

- Greedy vs Lazy:

<https://javascript.info/regexp-greedy-and-lazy>

- Debuggex:

<https://www.debuggex.com/>

- Regex 101

<https://regex101.com/>

- Regexp

<https://regextester.com/>